

CSS Layout

Next step after CSS basics
Display • Positioning • Flexbox • Grid • Responsive Design

Learning goals

By the end of this lecture, students should be able to:

- Explain why CSS layout is needed.
- Use display to change how elements are placed.
- Use position: static, relative, absolute, fixed, and sticky.
- Control overflow when content is too large.
- Understand float and clear.
- Build layouts with Flexbox.
- Build two-dimensional layouts with CSS Grid.
- Use media queries for responsive pages.

1. What is CSS layout?

Layout controls where elements appear and how much space they use.

- HTML creates the page content.
- CSS layout decides how that content is arranged.
- Layout can be one column, several columns, rows, cards, or a complete page.
- Modern CSS mainly uses Flexbox and Grid for layout.
- Other properties such as display and position are also important.

Simple layout example

```
.container {
  display: flex;
  gap: 20px;
}

.item {
  padding: 20px;
  border: 1px solid black;
}
```

Teacher tip: Start with the question: 'Where should this element go?'

2. The display property

The display property controls how an element is shown.

- display: block → starts on a new line and normally takes the available width.
- display: inline → stays in the same line and uses only needed width.
- display: inline-block → stays in a line but can have width and height.
- display: none → hides the element and removes its space.

Display examples

```
.box {
  display: block;
}

span {
  display: inline;
}

.card {
  display: inline-block;
  width: 200px;
}

.hidden {
  display: none;
}
```

Normal document flow

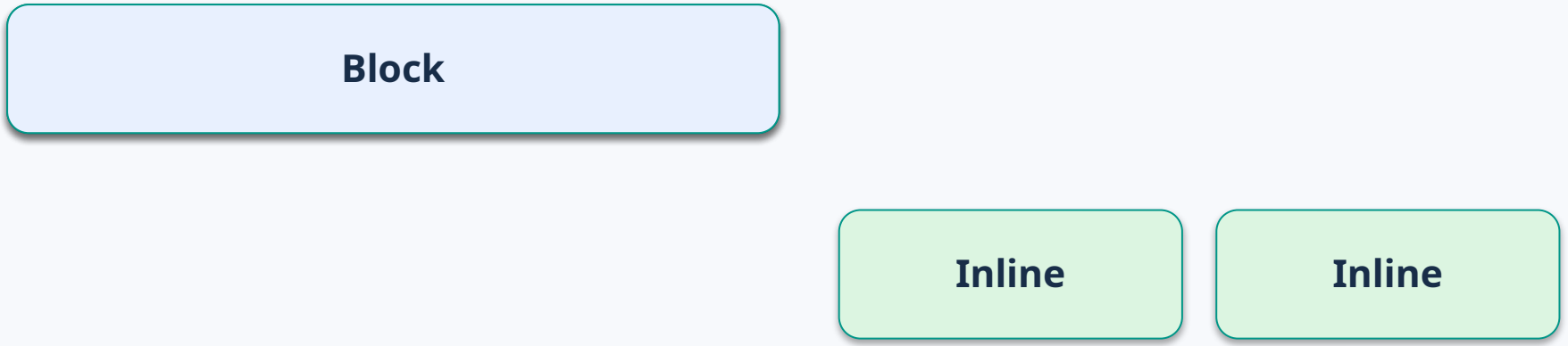
Positioning lets us control an element's location.

Normal document flow

- is the default layout system that browsers use to arrange elements on a web page.
- Unless you explicitly tell an element to behave differently (using properties like float, position: absolute, Flexbox, or Grid).
- The browser displays HTML elements in the exact order they appear in your source code.

3. Block vs. inline elements

Understanding this difference helps students understand layout.



- Block elements normally begin on a new line.
- Inline elements normally stay in the same line.
- Examples: div and p are block-level; span and a are commonly inline.
- CSS can change the display behavior.

4. Positioning in CSS

Positioning lets us control an element's location.

- static → normal document flow.
- relative → moves from its normal position.
- absolute → positioned relative to a positioned ancestor.
- fixed → stays in the same place in the browser window.
- sticky → acts like normal content until a scroll position is reached.

Position examples

```
.box {
  position: relative;
  top: 20px;
  left: 30px;
}

.button {
  position: fixed;
  right: 20px;
  bottom: 20px;
}
```

Static Position

Positioning lets us control an element's location.

- Every HTML element has a default CSS position property set to static.
- Static simply means **"let this element stay exactly where it naturally falls in the normal document flow."**
  Key characteristics of static elements:

- **No Offsets:** Edge offset properties—like top, bottom, left, and right—**have no effect**. You cannot nudge a static element using these coordinates.
- **No Depth Layering:** The z-index property **does not work** on static elements. They cannot be layered on top of or behind other elements using a custom stack depth.

Position examples

```
.box {
  position: relative;
  top: 20px;
  left: 30px;
}

.button {
  position: fixed;
  right: 20px;
  bottom: 20px;
}
```

5. Relative positioning

Relative positioning keeps the element's original space.

- The element stays in normal flow.
- top, right, bottom, and left can move it from its normal position.
- Other elements still behave as if it were in the original position.

Example

```
.box {
  position: relative;
  left: 30px;
  top: 10px;
}
```

7. Fixed and sticky positioning

These are useful for navigation bars and floating controls.

- fixed stays attached to the browser window while scrolling.
- sticky behaves normally, then sticks when its scroll limit is reached.
- Use them carefully so they do not cover important content.

Examples

```
<style>
      .box {
         background: lightblue;
         padding: 10px;
         margin: 5px;

         /* Static is the default. Top and left will be ignored. */
         position: static;
         top: 50px;
         left: 50px;
      }
  </style>
```

6. Absolute positioning

Absolute elements are removed from normal document flow.

- An absolute element is positioned using top/right/bottom/left.
- It is positioned relative to the nearest positioned ancestor.
- If no positioned ancestor exists, the page is used as the reference.
- Common use: placing a badge, icon, or label inside a card.

Card + badge example

```
.card {
  position: relative;
}

.badge {
  position: absolute;
  top: 10px;
  right: 10px;
}
```

8. z-index

z-index controls which positioned element appears in front.

- The z-index property controls the **three-dimensional depth (stacking order)** of elements along the virtual Z-axis (pointing toward or away from your face).
- It determines which elements overlap on top of others.
- The z-index property **only works on positioned elements**.
- If an element has the default position: static, z-index will be completely ignored.
- To use it, you must set the position to relative, absolute, fixed, or sticky.

Simple example

```
.front {
  position: absolute;
  z-index: 2;
}

.back {
  position: absolute;
  z-index: 1;
}
```

9. Overflow

Overflow controls what happens when content is larger than its box.

- visible → content can extend outside the box.
- hidden → extra content is clipped.
- scroll → scrollbars are shown.
- auto → scrollbars appear when needed.
- overflow-x and overflow-y can control each direction separately.

Example

```
.box {
  width: 250px;
  height: 100px;
  overflow: auto;
}
```

Teacher tip: Give the box more text than it can hold so students can see the scrollbar.

10. Float and clear

Float was widely used for layouts before Flexbox and Grid.

- float can move an element to the left or right.
- Text can wrap around a floated image.
- clear prevents an element from moving beside a floated element.
- Float is still useful for some simple content-wrapping cases.

Float example

```
.photo {
  float: left;
  margin-right: 15px;
}

.footer {
  clear: both;
}
```

Teacher tip: Tell students: for modern page layouts, learn Flexbox and Grid rather than using float as the main layout system.

11. Flexbox — the main idea

Flexbox is designed for arranging items in a row or column.

•Flexbox is a layout model for arranging items (horizontally or vertically) within a container, in a flexible and responsive way.

- Set the parent to display: flex.
- The parent becomes a flex container.
- Its direct children become flex items.
- Flexbox makes alignment and spacing much easier.
- It is especially useful for navigation bars, rows, columns, and cards.

Start Flexbox

```
.container {
  display: flex;
}
```

12. Flex direction

flex-direction decides whether items are arranged as a row or column.

- row → items go from left to right.
- column → items go from top to bottom.
- row-reverse and column-reverse reverse the direction.

Examples

```
.container {
  display: flex;
  flex-direction: row;
}

.column {
  display: flex;
  flex-direction: column;
}
```

Teacher tip: Change row to column during class and ask students what changed.

13. justify-content

justify-content controls alignment along the main axis.

- flex-start → items start at the beginning.
- flex-end → items move to the end.
- center → items are centered.
- space-between → equal space between items.
- space-around / space-evenly → distribute space around items.

Common example

```
.container {
  display: flex;
  justify-content:
    space-between;
}
```

14. align-items

align-items controls alignment on the cross axis.

- center is often used to vertically center items in a row.
- flex-start and flex-end move items toward opposite edges.
- stretch can make items fill the available cross-axis space.

Center items vertically

```
.container {
  display: flex;
  height: 200px;
  align-items: center;
}
```

Teacher tip: A common pattern is justify-content: center plus align-items: center.

15. Flex gap, wrap, and sizing

These properties help create clean card layouts.

- gap adds space between flex items.
- flex-wrap allows items to move onto a new line.
- flex can control how an item grows or shrinks.
- These features reduce the need for complicated margin rules.

Responsive card row

```
.container {
  display: flex;
  gap: 20px;
  flex-wrap: wrap;
}

.card {
  flex: 1 1 200px;
}
```

16. Complete Flexbox example

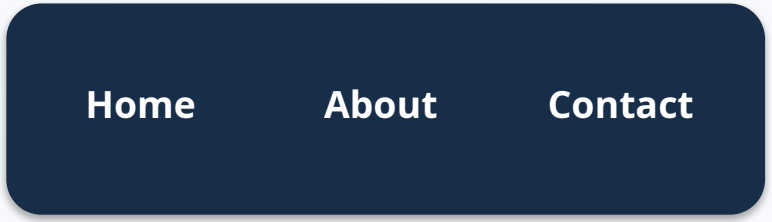
A simple navigation bar using Flexbox.

HTML + CSS

```
<nav class="nav">
  <a href="#">Home</a>
  <a href="#">About</a>
  <a href="#">Contact</a>
</nav>

<style>
.nav {
  display: flex;
  justify-content: space-between;
  align-items: center;
  gap: 20px;
  padding: 15px;
  background: navy;
}

.nav a {
  color: white;
  text-decoration: none;
}
```



Teacher tip: This is a good live-coding example: remove justify-content and let students observe the change.

17. CSS Grid — the main idea

Grid is designed for two-dimensional layouts: rows and columns.

- Set the parent to display: grid.
- grid-template-columns defines columns.
- grid-template-rows defines rows.
- gap creates space between grid items.
- Grid is excellent for page layouts and card galleries.

Three-column grid

```
.container {
  display: grid;
  grid-template-columns:
    1fr 1fr 1fr;
  gap: 20px;
}
```

18. Grid columns with repeat()

repeat() makes repeated columns easier to write.

- 1fr means one fraction of the available space.
- repeat(3, 1fr) creates three equal columns.
- minmax() can help create flexible responsive columns.

W3Schools-style Grid pattern

```
.grid {
  display: grid;
  grid-template-columns:
    repeat(3, 1fr);
  gap: 20px;
}

.responsive-grid {
  display: grid;
  grid-template-columns:
    repeat(auto-fit,
    minmax(200px, 1fr));
}
```

19. Placing Grid items

Grid items can span more than one column or row.

• grid-column controls the column position.

(grid-column: start-line / end-line;)

➢ places the item starting at column line 1 and ending at column line 3

• grid-row controls the row position.

(grid-row: start-line / end-line;)

➢ places the item starting at row line 2 and ending at row line 5

• span can make an item cover several columns or rows.

Grid placement examples

```
.item1 {
  grid-column: 1 / 3;
}

.item2 {
  grid-row: 2 / 4;
}

/* or */
.item3 {
  grid-column:
    span 2;
}
```

20. Flexbox or Grid?

Both are important; choose according to the layout.

Flexbox

Best for one main direction:
row OR column.

Examples:
navbars, button groups,
card rows.

Grid

Best for two dimensions:
rows AND columns.

Examples:
page layouts, galleries,
card grids.

21. Responsive Web Design

A responsive page adapts to different screen sizes.

- Desktop, tablet, and mobile screens have different widths.
- Flexible units and layouts help the page adapt.
- Flexbox and Grid can make layouts responsive.
- Media queries allow CSS rules to change at a certain screen width.

Media query example

```
/* Normal */
.container {
  display: grid;
  grid-template-columns:
    repeat(3, 1fr);
}

/* Smaller screens */
@media (max-width: 600px) {
  .container {
    grid-template-columns: 1fr;
  }
}
```

22. Responsive navigation example

A layout can change when the screen becomes smaller.

Example

```
.nav {
  display: flex;
  justify-content:
    space-between;
  gap: 20px;
}

@media (max-width: 600px) {
  .nav {
    flex-direction: column;
  }
}
```

- On a wide screen, links can appear in a row.
- On a small screen, the links can become a column.
- The browser applies the media-query rule only when the condition is true.

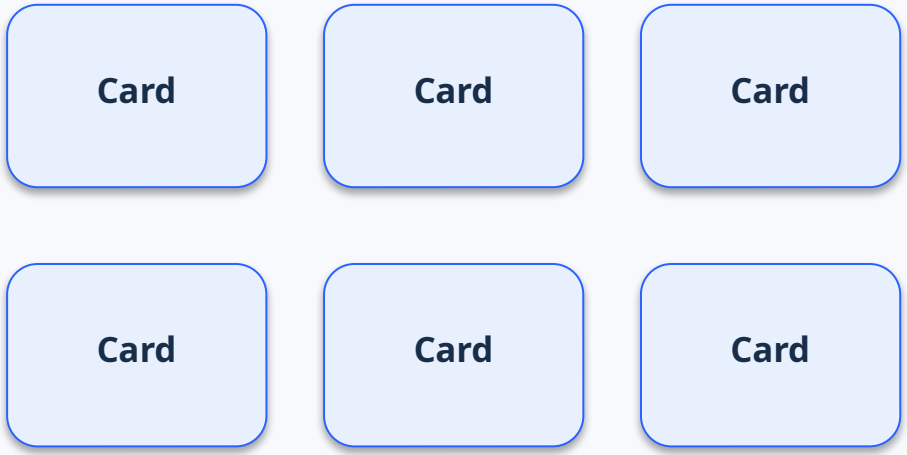
23. A simple responsive card layout

Combining Grid with a media query.

Complete example

```
.cards {
  display: grid;
  grid-template-columns:
    repeat(3, 1fr);
  gap: 20px;
}

@media (max-width: 700px) {
  .cards {
    grid-template-columns: 1fr;
  }
}
```



Teacher tip: On a mobile screen, the three columns become one column.

24. Layout mistakes beginners make

Most layout problems come from a small number of mistakes.

- Using absolute positioning for the whole page instead of normal layout tools.
- Forgetting that position: absolute is removed from normal flow.
- Using fixed widths that do not fit small screens.
- Not using gap, flex-wrap, or Grid when a layout needs them.
- Trying to fix every problem with margin.
- Writing a media query but placing it outside the CSS file or style block.

Teacher tip: When a layout breaks, inspect the parent first: display, width, direction, gap, and positioning.

25. Classroom practice: build a page layout

Students can complete this in 10–15 minutes.

- Create a header at the top.
- Create a main area with three cards.
- Use Flexbox for the navigation.
- Use Grid for the cards.
- Add gap between items.
- Make the cards become one column below 700px.
- Add a footer below the cards.

Starter CSS

```
.nav {
  display: flex;
  gap: 20px;
}

.cards {
  display: grid;
  grid-template-columns:
    repeat(3, 1fr);
  gap: 20px;
}

@media (max-width: 700px) {
  .cards {
    grid-template-columns: 1fr;
  }
}
```

26. Quick review

Ask students these questions before ending the lecture.

- What does display: flex do?
- What is the difference between relative and absolute positioning?
- When would you use fixed positioning?
- What does overflow: auto do?
- What is the main difference between Flexbox and Grid?
- What does gap do?
- Why do we use media queries?
- What happens to our card layout on a small screen?